

MTG Houdini: Multiplayer Life Counter Sync

Research Report — May 6, 2026

Technical feasibility analysis for real-time multiplayer syncing of life counters across player devices during in-person Magic: The Gathering games.

TL;DR Recommendation: Use **Supabase Realtime** (Broadcast + Presence) with room codes. It's already in your stack, requires ~50 lines of new code, works on all platforms, and is free at your scale. Every major MTG app with multiplayer uses this exact pattern.

1. Web Bluetooth API

DEALBREAKER — Not Viable

- **iOS Safari:** Not supported at all (any version). Hard blocker for cross-platform.
- **Firefox:** Not supported.
- **Chrome Android:** Supported, but designed for peripheral devices (heart monitors, etc.), not phone-to-phone.
- **Architecture issue:** GATT-based. Phones would need to act as both central and peripheral, which the API doesn't support well.
- **PWA standalone mode:** Inconsistent behavior even on Chrome Android.

Kill reason: Zero iOS Safari support makes this a non-starter for any cross-platform mobile app.

2. WebRTC Data Channels (PeerJS)

VIABLE — But Complex

- **Browser support:** ~96% global. iOS Safari 11.2+, Android Chrome, Firefox, Edge.

- **How it works:** Peer-to-peer data channels. Once connected, data flows directly between devices with sub-10ms latency on the same Wi-Fi.
- **Signaling server required:** Needs a server to broker initial connections (SDP offers/answers, ICE candidates). PeerJS Cloud offers this for free, or Supabase Realtime could serve as the signaling channel.
- **NAT traversal:** Needs STUN servers (Google provides free ones). TURN servers needed as fallback — costs money but rarely needed on same Wi-Fi.
- **Mesh topology:** For 6 players, you'd need 15 peer connections (full mesh) or a star topology where one host relays.
- **Cost:** Free with PeerJS Cloud + Google STUN.

Downside: Connection setup takes 2-5 seconds. Managing disconnects/reconnects and state reconciliation across a mesh adds significant complexity compared to server-mediated approaches.

3. WebSockets (Server-Mediated)

VIABLE — But Hosting Challenge on Vercel

- **Browser support:** 96%+ universal. No compatibility issues on any platform.
- **Vercel limitation:** Vercel Serverless Functions do not natively support persistent WebSocket connections. You'd need an external provider.
- **Latency:** 20-100ms round-trip through server. Adequate for life counter updates.
- **Requires internet:** Yes, always.

Service	Free Tier	Cost at Scale	Notes
Supabase Realtime	200 connections, 2M msgs/mo	\$2.50/M messages	Already in stack
Ably	200 connections, 6M msgs/mo	\$2.50/M messages	Robust, good docs
Pusher	200 connections, 200K msgs/day	Per-message pricing	Popular, vendor lock-in
Socket.io on Fly.io	Fly.io free tier	~\$5/month	Self-managed

4. Supabase Realtime (Broadcast + Presence)

STRONGLY RECOMMENDED — Best Fit

This is the clear winner. The app already uses Supabase and `@supabase/supabase-js` v2.103.0 already includes the Realtime client. Zero new dependencies required.

How It Works

- **Broadcast:** Send arbitrary JSON messages to all subscribers of a named channel. Perfect for life change events and game actions.
- **Presence:** Track shared state per user. Each client calls `track()` with their state, and all clients receive `sync`, `join`, and `leave` events. Perfect for "who is in this game" and "what are their life totals."

Example Code

```
const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY)

// Join a game room
const channel = supabase.channel('game-room-A7X3KM')

// Track this player's state via Presence
channel.subscribe(async (status) => {
  if (status === 'SUBSCRIBED') {
    await channel.track({
      playerId: 'player-1',
      name: 'Dan',
      life: 40,
      poison: 0,
      commanderDamage: {}
    })
  }
})

// Listen for all players' state changes
channel.on('presence', { event: 'sync' }, () => {
  const state = channel.presenceState()
  // state contains all players and their life totals
})
```

```
// Broadcast discrete events (animations, game reset)
channel.on('broadcast', { event: 'life-change' }, (payload) => {
  // { playerId: 'player-2', life: 38, delta: -2 }
})
```

Key Characteristics

Aspect	Detail
Browser Support	Universal (WebSocket-based). iOS Safari, Android Chrome, all PWA modes.
Latency	~50-150ms. Imperceptible for life counter taps.
Complexity	LOW. ~50 lines of client code for a working sync.
Cost	Free tier: 200 concurrent connections, 2M msgs/month. A 6-player game uses ~100 messages per session.
New Dependencies	None. Already have <code>@supabase/supabase-js</code> .
Offline	No — requires internet. Standard for same-table play in 2026.

5. Room Code / QR Code Lobby System

RECOMMENDED — Pairs with Supabase Realtime

Room Codes

- Generate 4-6 character alphanumeric codes client-side (e.g., "A7X3KM").
- $36^6 = \sim 2.2$ billion possible codes. Collision risk is negligible.
- No server-side registry needed — the Supabase channel name *is* the room.
- Subscribing to `life-counter-A7X3KM` puts you in that room.

QR Code (Nice-to-Have)

- Display QR encoding `https://mtghoudini.com/life?room=A7X3KM`.

- Use `react-qr-code` or `qrcode` npm package for generation.
- Scanning opens the app and auto-joins the room.
- Not essential — typing 4-6 characters is fast enough for same-table play.

Player Flow

1. Host taps "**Create Room**" → generates code, displays it large on screen.
2. Other players tap "**Join Room**" → type the code (or scan QR).
3. All subscribe to the same Supabase channel.
4. Presence `sync` event populates the player list with live life totals.

6. Local Network Approaches (mDNS, Wi-Fi Direct)

NOT VIABLE — No Web APIs

- **mDNS / Bonjour:** No web API exists. Native apps only.
- **Wi-Fi Direct:** No web API. Android-only native API.
- **Web NFC:** Chrome Android only. Not on iOS. Could tap-to-share a room code, but can't do continuous sync.

The web platform has no local-network peer discovery APIs that work cross-platform. All "local" solutions ultimately require an internet server.

What Existing MTG Apps Do

- **Carbon (MTG Life Counter):** Server-based sync with room codes. Requires internet.
- **Lifelinker:** Cloud-based rooms with join codes.
- **MTG Familiar:** No multiplayer sync.

The industry standard is room codes + cloud sync. No major MTG app uses Bluetooth or peer-to-peer.

Comparison Matrix

Approach	iOS Safari	Complexity	Latency	Cost	Offline	Verdict
Web Bluetooth	No	High	Low	Free	Yes	Dealbreaker
WebRTC (PeerJS)	Yes	High	Very Low	Free*	Partial	Backup option
WebSockets (3rd party)	Yes	Medium	Low	Free tier	No	Viable
Supabase Realtime	Yes	Low	Low	Free tier	No	Recommended
Local Network	No	N/A	N/A	N/A	Yes	Not viable

Implementation Roadmap

If you decide to build this with Supabase Realtime, here's what's needed:

1. **Client-side Supabase instance** — New file `src/lib/supabase/client.ts` using `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` environment variables.
2. **`useMultiplayerRoom` hook** — Wraps channel subscription, Presence tracking, and Broadcast events.
3. **Room create/join UI** — Modal in the life counter setup page with code input + display.
4. **Sync integration** — Modify existing life counter state to optionally broadcast changes via the room channel.
5. **Supabase Realtime policy** — Enable Broadcast/Presence for anonymous or authenticated users (Supabase dashboard setting).

Estimated effort: 2-3 development sessions. No new infrastructure, no new dependencies, no additional cost.
